

# ELEC 475 Lab 4

Names: Sage Franco, Thomas Wilkinson

Student Numbers: 20280229, 20269155

Date: December 2<sup>nd</sup>, 2025

# Introduction

## **Explain the motivation and structure of CLIP.**

CLIP (Contrastive Language-Image Pre-training) is a visual-language model that relates images to natural language text. The motivation of CLIP was to overcome the limitation of traditional image classification models that require large manually labelled datasets with fixed list of categories. Instead, CLIP uses image-text pairs taken from the internet, where the labels are natural language captions describing the content of the images.

CLIP includes an image encoder for the images and a text encoder for the text to encode image and text embeddings in a shared 512-dimensional latent space. The goal during training is to reduce the distance between the matching image and text embeddings in the latent space. It uses infoNCE loss to pull the matching image-text pairs closer and push the non-matching pairs further apart.

By having both text and image embeddings in a shared latent space, CLIP supports zero-shot classification and cross-model retrieval. Zero-shot classification allows new images into categories that were not seen in training through semantic similarity in the shared latent space. Cross-model retrieval allows for searching in both directions, text to image and image to text. By inputting a new text caption, you can get the closest images in the embedding space, and by inputting a new image, you can get the closest captions to that image.

# Methodology

## **Summarize your model and training design.**

The text preprocessing script uses a text encoder to create the text embeddings of the train and val splits of the COCO2014 dataset. The text encoder is a pretrained Transformer from OpenAI's CLIP model (openai/clip-vit-base-patch32). The CLIP model and CLIP processor are loaded and set to evaluation mode. During text encoding, the CLIP processor handles tokenization and padding and the processed captions get converted to embeddings using the CLIP model's `get_text_features` method. The embeddings are then stored in train and val .pt files as tuples of image filename and tensor of shape `num_captions x 512`.

The dataset setup file includes the `CocoClipDataset` class. The `_init_` function loads the preprocessed cache of COCO2014 image\_filename-caption embeddings and saves them to a container called `data`. The `_len_` function outputs the total number of image-text pairs in the cached dataset by checking the length of `data`. The `_get_item_` function loads an image from the disk using its `image_filename` stored in `data`, converts it to RGB, applies transformations and random sampling, and returns a tuple of the image (tensor 3x224x224) and the caption embedding (tensor 512). Image Normalization uses the standard CLIP mean (0.48145466, 0.4578275, 0.40821073) and standard deviation (0.26862954, 0.26130258, 0.27577711). The image transformations had option to include augmentation. Without augmentation, it resizes and center crops the images to 224x224, converts it to a tensor and normalizes it. With augmentation, it does the same, but also adds random resizing/cropping, random horizontal flip of  $p=0.5$ , random rotation up to 15 degrees maximum, colour jitter, and random grayscale of  $p=0.2$ . Although the COCO dataset provided 5

captions per image, the group used a random sampling strategy where for each epoch, for every image, one of the 5 corresponding captions was randomly selected. This enabled the group to use the standard InfoNCE loss asked for in the lab while maximizing linguistic diversity across epochs, essentially helping the model better generalize to the training set.

The image encoder model uses ResNet50 architecture pretrained on ImageNet. The projection head added consists of a two-layer MLP which maps the ResNet features to the 512-dimensional CLIP embedding space. The first layer goes from 2048 to 2048-dimensional feature vector, followed by a GELU activation function, and the second layer goes from 2048 to a 512-dimensional feature vector. During a forward pass, the input image passes through this ResNet50 backbone and the projection head, resulting in an output embedding that is compatible with the CLIP text embeddings outputted during the text preprocessing.

The training script uses contrastive training to align images and captions in a shared embedding space, where matching image-text pairs are close together and mismatched pairs are far apart. It loads the full COCO2014 dataset training split and a subset of 15,000 validation images are used for evaluation during training. The data loaders use `num_workers=12`, `pin_memory=True`, and shuffles only the training set, with a large batch size of 734. The ResNet50 image encoder is initialized along with the logit scale, which is a learnable temperature parameter used to control the sharpness of similarity scores. The InfoNCE loss function normalizes the image and text embeddings, computes cosine similarity, and computes bidirectional cross entropy loss. The AdamW optimizer is used with a learning rate of  $1e-5$  for the pretrained backbone and  $5e-4$  for the projection head and logit scale, along with a weight decay of 0.05. The learning rate scheduler used is Cosine annealing which gradually reduces the learning rate each epoch. GradScaler is used to prevent loss scaling issues and gradient clipping with a `max_grad_norm` of 1.0 is used to prevent exploding gradients (`torch.nn.utils.clip_grad_norm_`). `Torch.autocast()` is also used to improve training speed and reduce GPU memory usage. This script is set to train for 60 epochs, and the train and validation loss are logged after each epoch. The best checkpoint is saved at the end, including the model weights, optimizer and scheduler state, epoch number, and learned logit scale.

## Results

The group conducted an ablation study using two modifications, (1) data augmentation and (2) model improvements, including dropout and batch normalization. Training was done with no modifications, with only data augmentation, with only model improvements, and with both data augmentation and model improvements. These were all trained on Google Colab using the A100 GPU, which took around 2.5 hours for each. Each time, the model was trained for 60 epochs; however, some plots show less epochs as loss logging and model weights were only saved if validation loss was lower than the previous lowest one. The loss plots for each of the four training attempts can be seen as Figures 1 to 4 below. If the team was able to reproduce the experiment, they would still only save the model weights with the lowest validation loss, but they would continue plotting the loss, even if it remained higher than the lowest recorded one. However, these plots still provide all the necessary information for model comparison, especially considering that if the loss stopped plotting at an earlier epoch, the continued validation loss would have remained steady at around the same loss while the training loss would have continued to overfit.

The group did not observe many issues as they were prevented in the training script. As divergence is usually caused by exploding gradients, gradient clipping (`torch.nn.utils.clip_grad_norm_`) was added to prevent the gradients from becoming too large. Instability is when a model struggles to converge and the loss keeps jumping up and down. For CLIP, a small batch size can cause this to happen as there are not enough negatives to learn from. To prevent instability, the group used the largest batch size possible which was 734 when training using a Google Colab GPU with 40GB of RAM. Using a learning rate scheduler also helped to prevent instability by gradually reducing the learning rate, helping the model to converge.

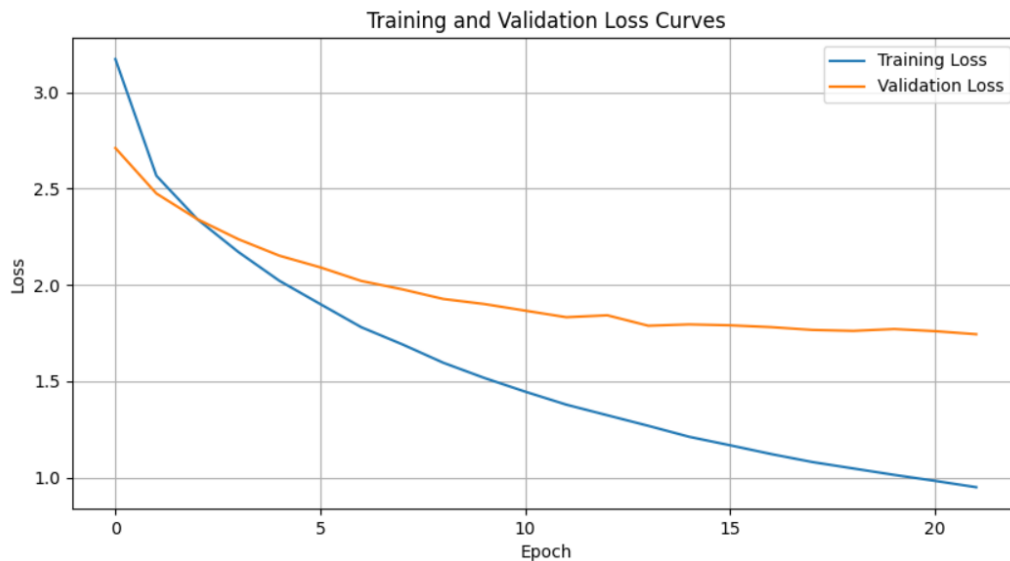


Figure 1: Loss Plot for training with no modifications.

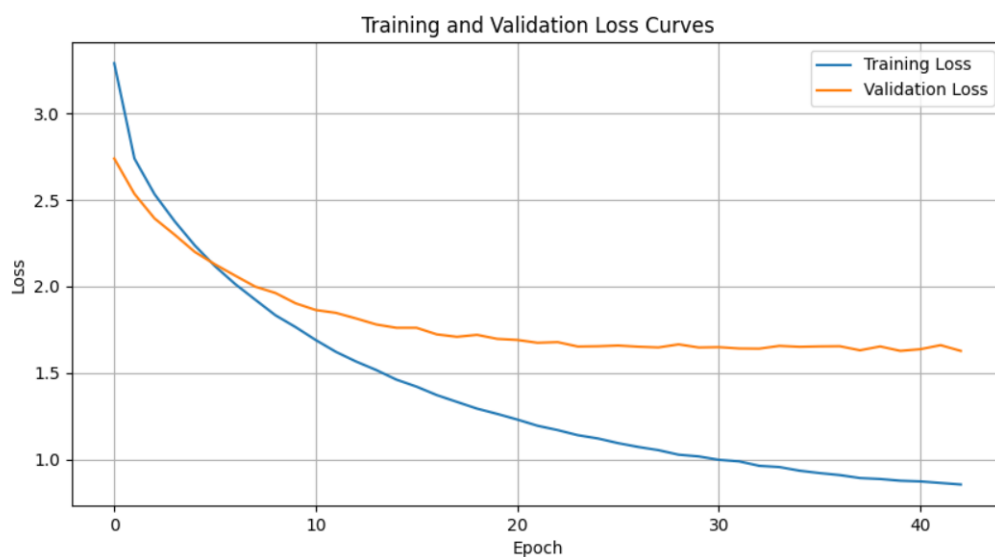


Figure 2: Loss Plot for training with modified model architecture (Batch Normalization and Dropout).

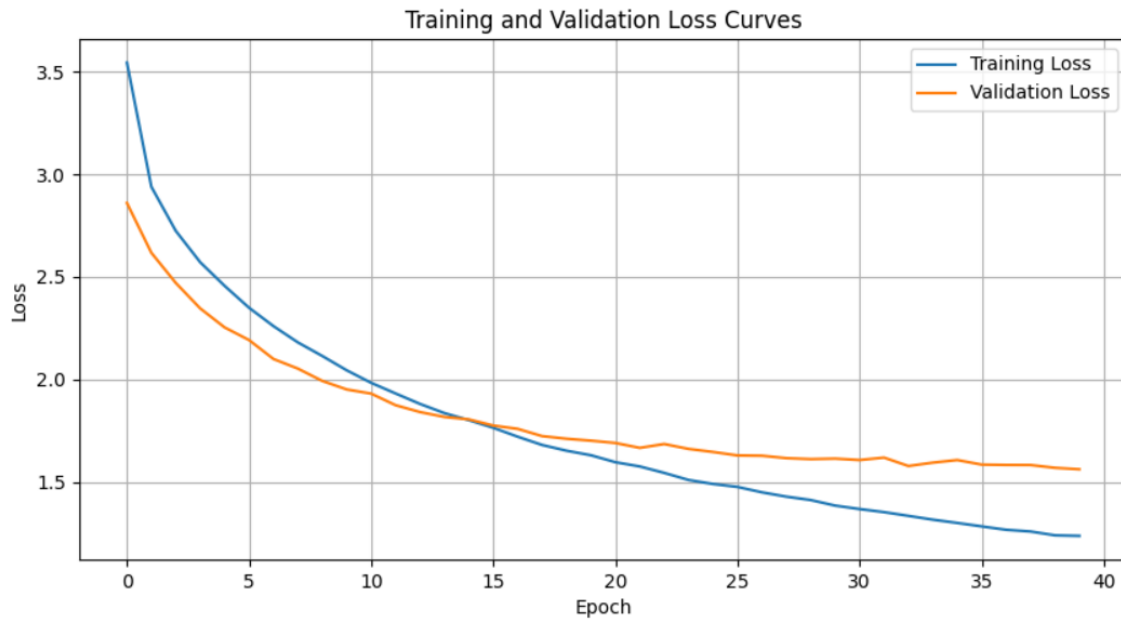


Figure 3: Loss Plot for training with data augmentation.

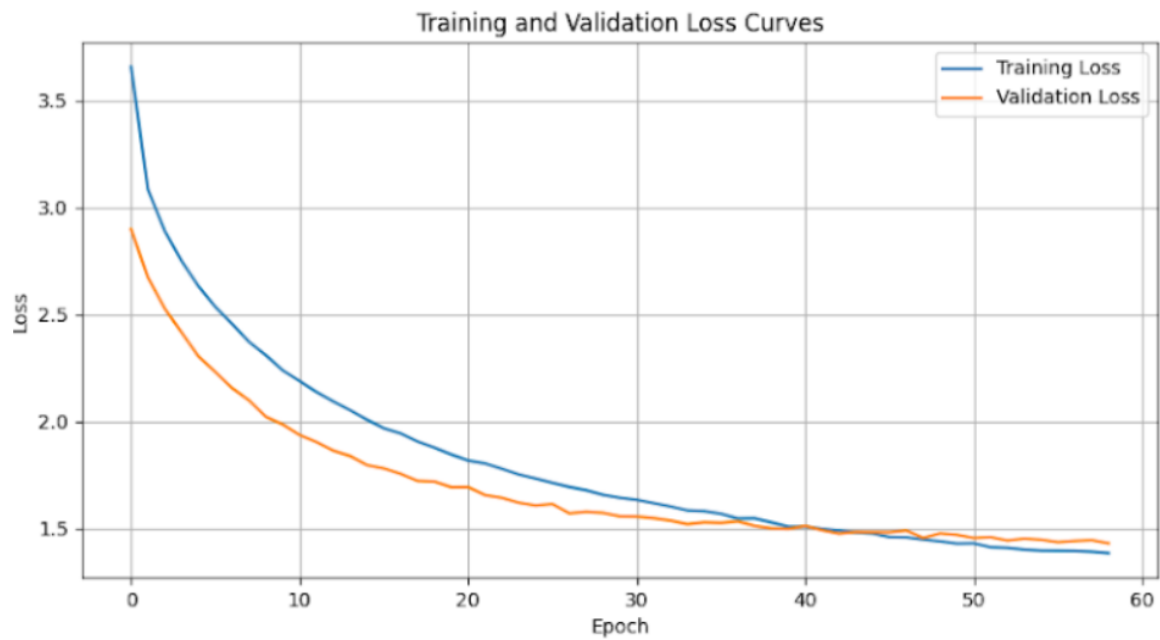


Figure 4: Loss Plot for training with model modifications (Batch Normalization and Dropout) and data augmentation.

The following figures show the Recall@K and retrieval visualization examples for each version outputted by our testing script. For top five image retrievals the top five captions used where building, a group of animals, swimming, and parking lot. For the zero-shot classification the image could be classified as either a person, an animal, a landscape, food, or a vehicle.

**Recall@K Scores:**

|               |      |  |        |
|---------------|------|--|--------|
| Image-to-Text | R@1  |  | 23.16% |
| Image-to-Text | R@5  |  | 51.30% |
| Image-to-Text | R@10 |  | 63.84% |
| Text-to-Image | R@1  |  | 24.38% |
| Text-to-Image | R@5  |  | 51.64% |
| Text-to-Image | R@10 |  | 64.62% |

Figure 5: Recall scores for training with no modifications.

|               |      |  |        |
|---------------|------|--|--------|
| Image-to-Text | R@1  |  | 25.26% |
| Image-to-Text | R@5  |  | 53.12% |
| Image-to-Text | R@10 |  | 66.96% |
| Text-to-Image | R@1  |  | 25.66% |
| Text-to-Image | R@5  |  | 55.16% |
| Text-to-Image | R@10 |  | 68.24% |

Figure 6: Recall scores for training with data augmentation.

|               |      |  |        |
|---------------|------|--|--------|
| Image-to-Text | R@1  |  | 25.72% |
| Image-to-Text | R@5  |  | 52.98% |
| Image-to-Text | R@10 |  | 65.60% |
| Text-to-Image | R@1  |  | 25.80% |
| Text-to-Image | R@5  |  | 53.30% |
| Text-to-Image | R@10 |  | 66.04% |

Figure 7: Recall scores for training with modified architecture.

|               |      |  |        |
|---------------|------|--|--------|
| Image-to-Text | R@1  |  | 27.44% |
| Image-to-Text | R@5  |  | 56.56% |
| Image-to-Text | R@10 |  | 68.94% |
| Text-to-Image | R@1  |  | 28.46% |
| Text-to-Image | R@5  |  | 57.72% |
| Text-to-Image | R@10 |  | 70.20% |

Figure 8: Recall scores for training with both modifications.

**Text to image retrieval visualizations:**

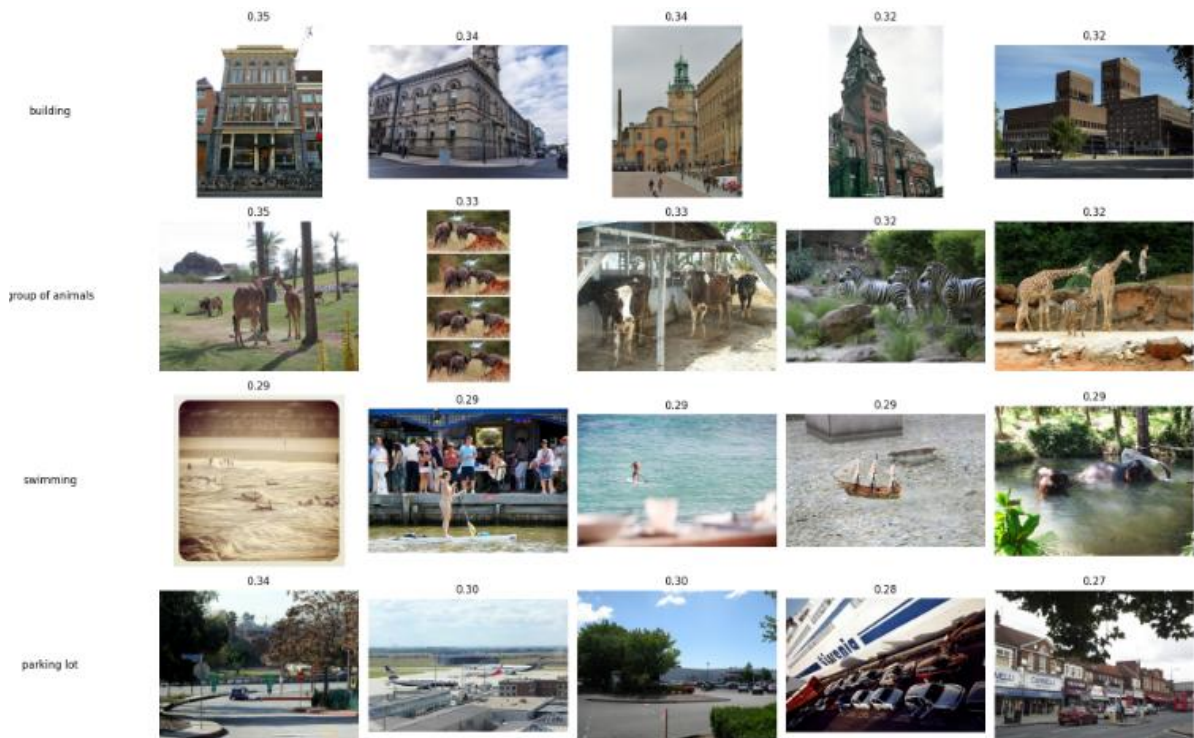


Figure 9: Top five image retrievals for training with no modifications.



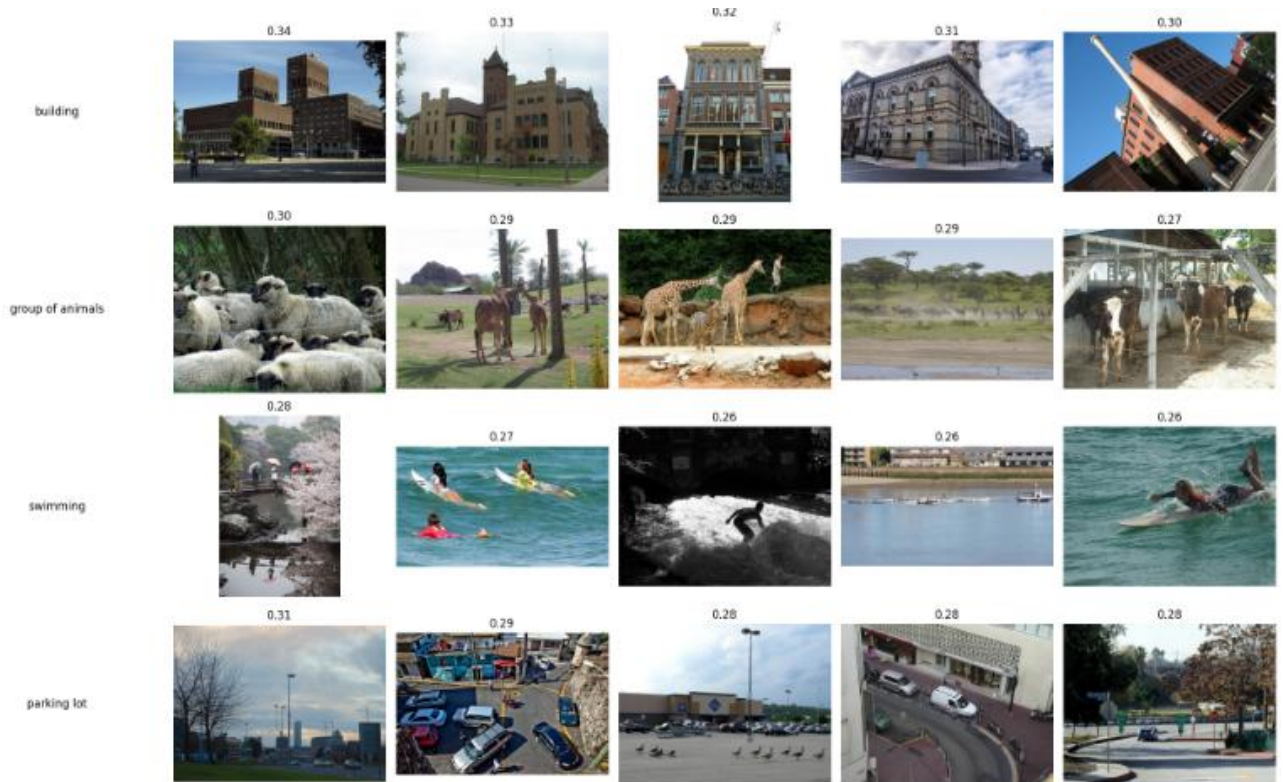


Figure 10: Top five image retrievals for training with data augmentation.

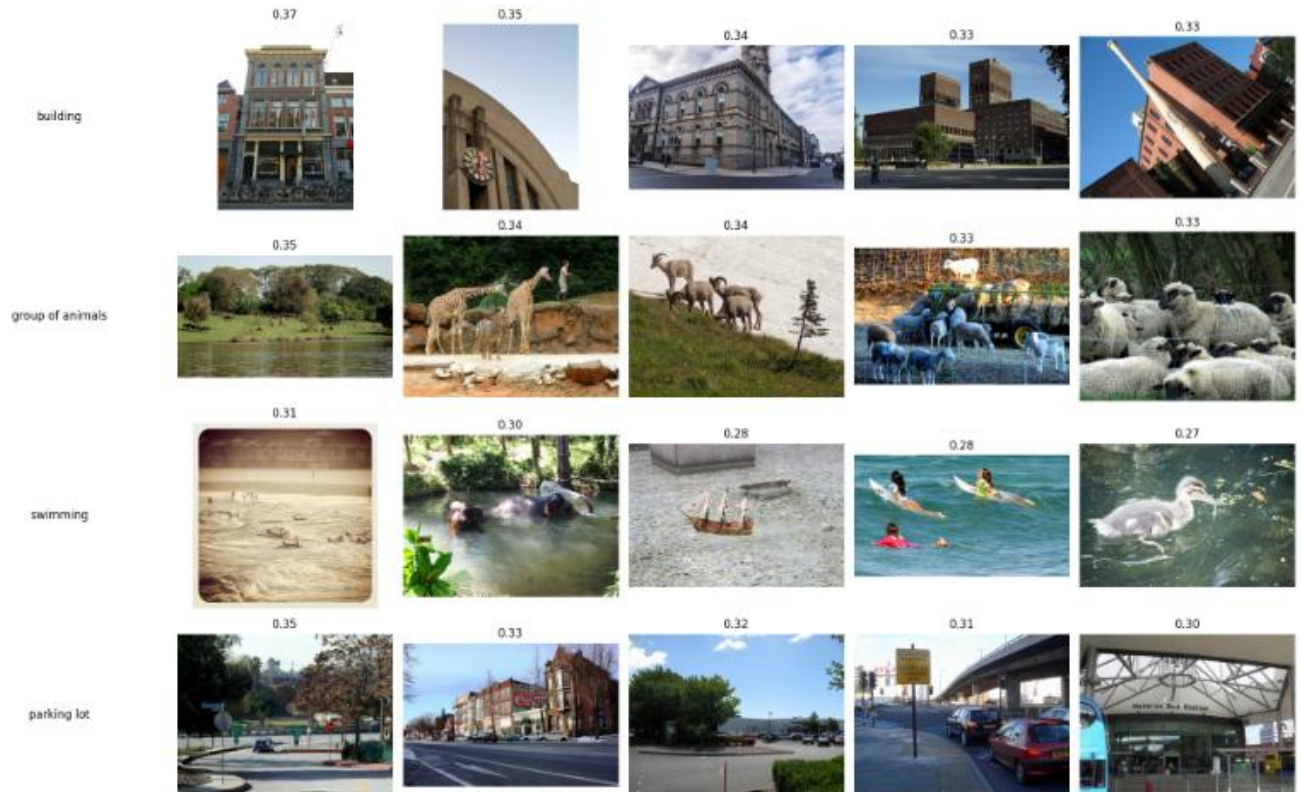


Figure 11: Top five image retrievals for training with modified model architecture.

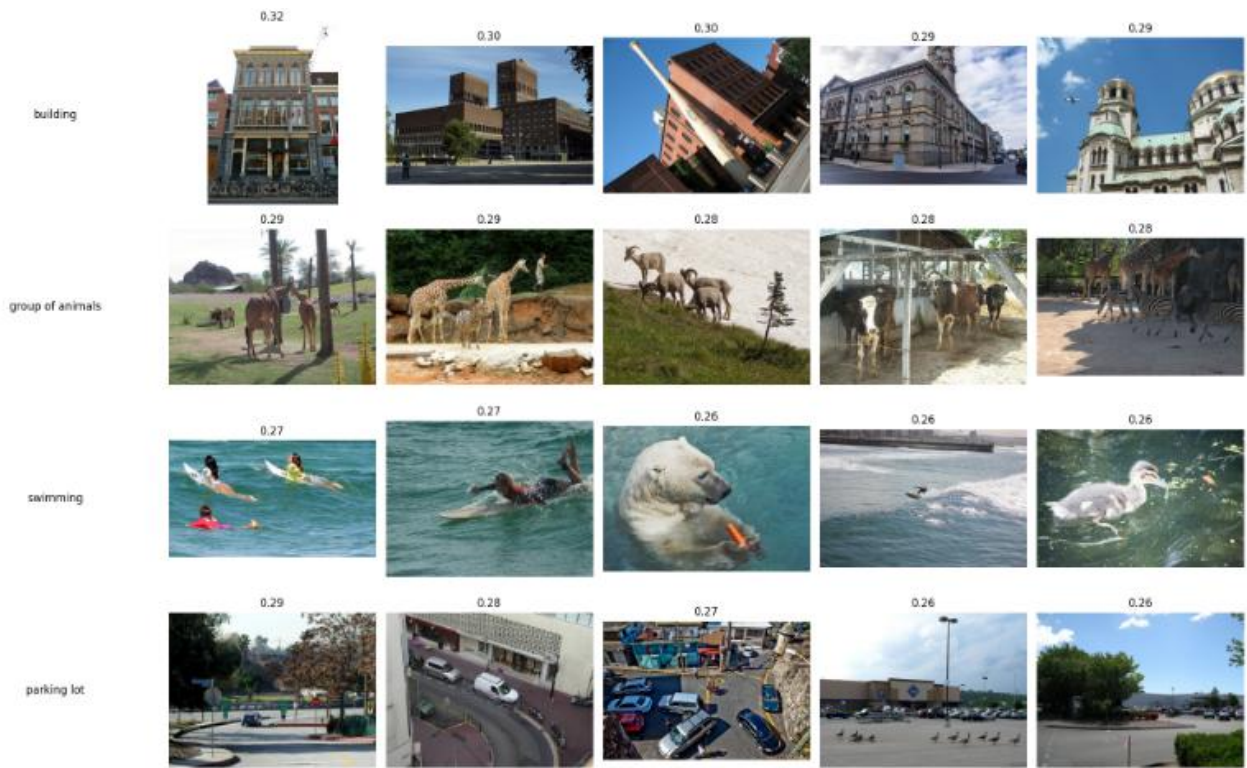


Figure 12: Top five image retrievals for training with both modifications.

### Image to text visualizations:

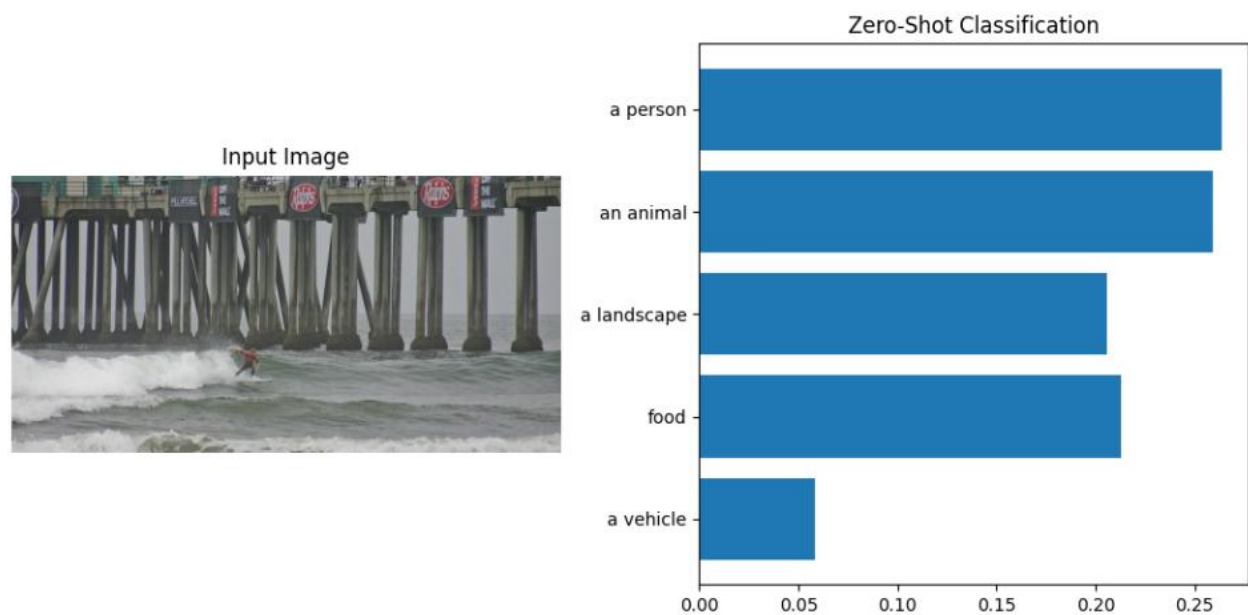


Figure 13: Zero shot classification for training with no modifications.



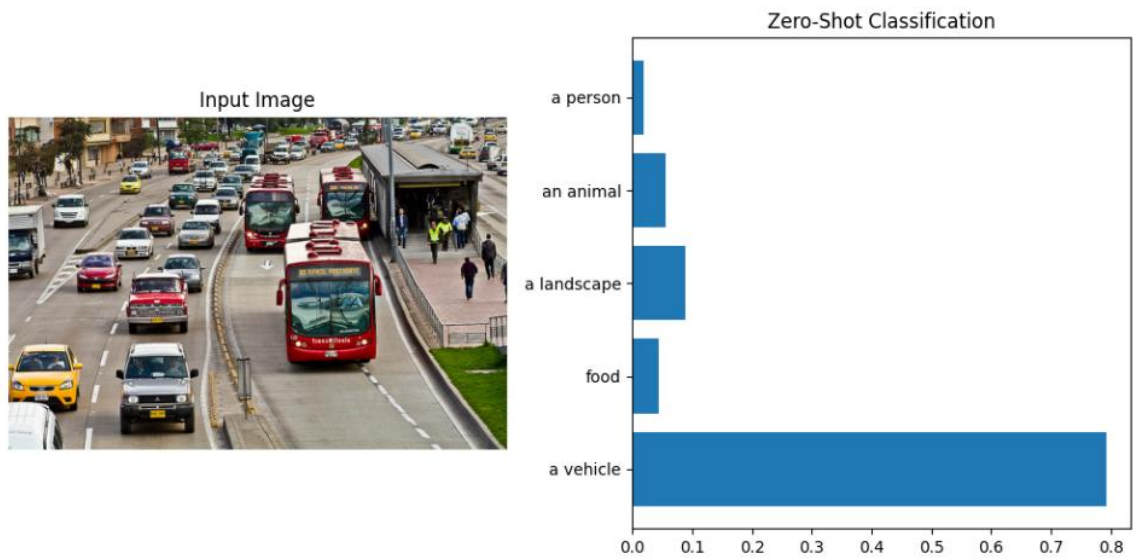


Figure 14: Zero shot classification for training with data augmentation.

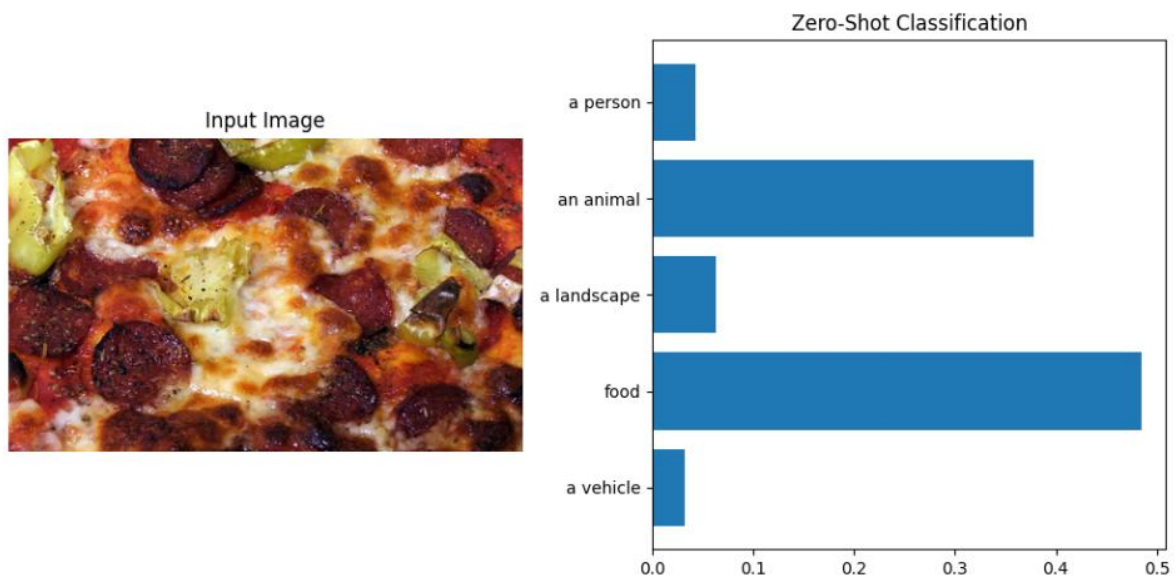


Figure 15: Zero shot classification for training with modified model architecture.

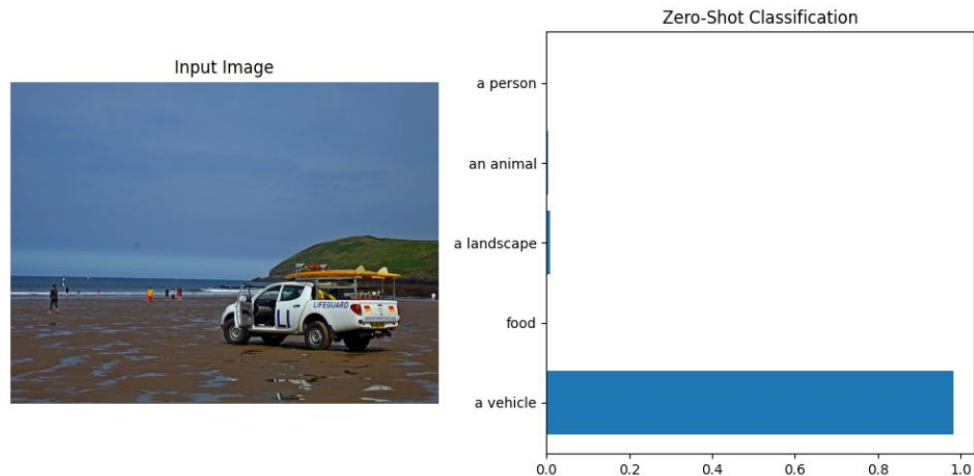


Figure 16: Zero shot classification for training with both modifications.

## Discussion

### Explain the mathematical intuition behind the CLIP loss (anchors, positives, and negatives)

In CLIP loss, each image is considered an anchor. Its matching captions are positives, and all other captions are negatives. Within each batch, the contrastive objective is to pull all matching image-text pairs together in the shared latent space and push all non-matching image-text apart. The group decided on a higher batch size which creates more negatives and improves training.

In each batch, a cosine similarity matrix is created for all image embeddings and all text embeddings, which creates an  $N \times N$  matrix of each image-text pair's cosine similarity. The SoftMax function is applied to every row of the matrix, which creates a probability distribution over the possible matches. The InfoNCE loss compares the probability of the correct match to the highest probability incorrect match. This is done in both directions, image-text and text-image, and then both losses are averaged. Over time, the InfoNCE loss encourages the probability of the true match to be the highest of all possible options and ensures that the text and image embeddings are jointly aligned in the shared latent space.

### Interpret results and analyze model behavior.

The loss plots are as expected: the loss plot with no modifications began overfitting earliest and has the largest validation loss (1.745), the loss plot with only modified model architecture overfits a little less and has a lower validation loss (1.627), the loss plot with only data augmentation has even less overfitting due to the added variability and has a lower loss than the modified model architecture (1.562), and the loss plot with both modifications does not overfit, generalizes well and has the lowest validation loss out of all of them (1.429). The loss plot results align with the accuracy of the testing results where recall is calculated, image retrieval is tested, and zero shot classification is tested.

Recall@K is a ranking metric used for image-text and text-image retrieval. It is the percentage of correct matches found in the top K results. For example, for image-text retrieval, the recall@5 is the

percentage of images that had their correct caption show up in the top 5 results. As expected, the recall@K scores gradually get higher from no modifications to one modification to both modifications. For both text-image and image-text, recall@1 ranges from 23.16% to 28.46%, recall@5 ranges from 51.3% to 57.72%, and recall@10 ranges from 63.84% to 70.2%.

Image retrieval converts the query text to a text embedding in the shared embedding space and calculates the cosine similarity between the text embedding and all other image embeddings. The images with highest cosine similarity are returned. For image retrieval, the group queried for “buildings”, “a group of animals”, “swimming”, and “parking lots”. All model versions correctly retrieved 5 images of “buildings” and “groups of animals”. For “swimming”, the models with no modifications or just one modification returned pictures of water, including boats in the water or someone paddleboarding, but not necessarily swimming. The model with both modifications has higher accuracy as it returned animals and people swimming in the water. The same occurs for “parking lots” where the model with no modifications or one modification sometimes returned images of roads, where the model with both modifications always return parking lots. For all models, they return accurate or near-accurate results, with the model with both modifications achieving the most accuracy, particularly in the details of the images.

Zero shot classification is the classification of images when their captions were never seen in training. To find the correct match, the image and class labels are converted to image and text embeddings, and the cosine similarities are computed. Due to the learned semantic alignment between text and images during contrastive training, the correct text label should have the highest similarity score. The zero shot classification test cases gradually get better from no modifications to model architecture modification to data augmentation to both modifications. With no modifications, the image is a person surfing, and the guess with the highest probability is correct (person), but it is not very certain as all 5 of the options have a high probability. With the modified model architecture, the image is a closeup of a pizza, where the highest probability guess (food) is still accurate, but there is another high probability guess (animal) that is incorrect. With data augmentation, the image is traffic on a road, and there is only one high probability which is accurate (vehicle), while there are small probabilities for all the other classes. For both modifications, the picture is a vehicle on the beach, where the model is almost 100% certain that is a vehicle and the other small percentage goes to landscape, and both are correct.

**Reflect on your use of LLMs or chatbots during the lab; e.g., what queries were helpful and how you validated their outputs. Include a link to your conversation.**

For this lab the team primarily used Gemini 3.0. The team found it helpful to first give the model context relating to the project by inserting the assignment instructions or by inserting related scripts. For example, if the dataset class had already been completed, providing the model with that code and then asking it to write the testing or training scripts ensured the model understood the task at hand and how it related to or impacted the rest of the project. The team also tried comparing Gemini’s responses with several different models (ChatGPT and Claude) to make sure they were correct and to see if better responses could be generated from different models. However, for this project, Gemini provided the best results, leading the team to use it for the entire project. Additionally, Gemini’s responses were also validated using validation scripts. For example, checking if the tensor outputted by the Resnet model was the correct shape, or checking if the

dataset loaded the data correctly by plotting the transformed images and caption pairs side by side (the code for these checks can be found in the `resnet50_model.py` and `DatasetSetup.py` scripts under the model and dataset classes). For training and testing, checking Gemini's responses was simply just seeing if the plotted loss decreased in an expected way and if the zero-shot classification and image retrievals were accurate. The team also learned the code that was given by chat and asked it for clarifications on some of the code that was more difficult to understand, this process also proved to help identify when Gemini was hallucinating. However, Gemini hallucinated very rarely when given prompts with sufficient context and descriptions.

Conversation getting first versions of `PreprocessTextEmbeddings.py`, `DatasetSetup.py` and `verify_dataset.py`: <https://gemini.google.com/share/7463a0315cfc>

Conversation getting first versions of `train.py` and `resnet50_model.py` scripts: <https://gemini.google.com/share/9ab40033c9a3>

Conversation getting first versions of the `test.py` script: <https://gemini.google.com/share/cb15528162ab>

Conversation modifying test and preprocessing scripts: <https://gemini.google.com/share/d7a86fb10c57>

Conversation asking about the best way to train the model with 5 captions and 1 image ratio: <https://gemini.google.com/share/d63da19bb55c>

Conversation getting script to verify dataset setup class plotting transformed images with caption pairs: <https://gemini.google.com/share/d49b19b88f41>

Conversation getting data augmentation and model improvements: <https://gemini.google.com/share/161115ecda76>

## Conclusion

In this lab, the group trained a CLIP-style model on the COCO2014 dataset. The group learned how to construct the model using a pretrained transformer for the text encoder and a resnet50 backbone and gained experience optimizing training on large-scale datasets. By implementing the CLIP model along with the training and testing scripts, the group gained a thorough mathematical understanding of contrastive learning using InfoNCE loss, logit-scale temperature learning, recall@K evaluation, zero shot classification, and how image and text embeddings are aligned and compared in the shared semantic space. The group also learned how to optimize prompts for LLMs and identify possible hallucinations.

Potential improvements include extending training time and increasing the batch size using a higher RAM GPU. In addition, further experimentation, such as using stronger augmentation, improving model architecture, or refining parameters, may lead to better results. Overall, this lab allowed the group to gain hands-on experience with CLIP-style models, contrastive learning, and multiple CLIP evaluation techniques.